

**A PROJECT REPORT
ON
VIDEO BASED ABNORMAL DRIVING BEHAVIOR DETECTION
VIA DEEP-LEARNING FUSIONS**

SUBMITTED

TO

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, ANANTAPUR

(IN PARTIAL FULLFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE IN)

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Sk. Manjunnisa	(21HN1A0550)
P. V. Meghana	(21HN1A0565)
Y. Pavan Kalyan	(21HN1A0563)
V. Tanuja	(21HN1A0561)
T. Mahendra	(21HN1A0557)

**UNDER THE VALUABLE GUIDANCE OF
MR. D. V. SATHISH, M. Tech, MBA, LLM, BLiSc, (PhD)
Assistant Professor**



ANDHRA ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA

Anantapur

3-76-2 Atmakur, SPSR Nellore (Dt), Andhra Pradesh, India-524322

(2021-2025)



Website: www.aecn.in

Email: info@aecn.in

Ph:9000377399

ANDHRA ENGINEERING COLLEGE

Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA Anantapur 3-76-2 Atmakur,
SPSR Nellore (Dt), Andhra Pradesh, India-524322

BONAFIDE CERTIFICATE

This is to certify that the project work entitled “**VIDEO BASED ABNORMAL DRIVING BEHAVIOR DETECTION VIA DEEP-LEARNING FUSIONS**” is a Bonafide record done by **Sk. Manjunnisa (21HN1A0550), P. V. Meghana (21HN1A0565), Y. Pavan Kalyan (21HN1A0563), V. Tanuja (21HN1A0561), T. Mahendra (21HN1A0557)** In the Department of Computer Science & Engineering Andhra Engineering College, Atmakur and is submitted to Jawaharlal Nehru Technological University, Anantapur in the partial fulfilment for the award of **B.Tech degree in Computer Science & Engineering**. This work has been carried out under my supervision.

Mr. D. V. Sathish
Project Guide
Dept of CSE
AECN, Atmakur

Mr. O. Devanand
Associate Professor and Project Guide
Dept of CSE
AECN, Atmakur

Submitted for the Viva-Voce Examination held on _____

Internal Examiner

External Examiner

(2021-2025)

ACKNOWLEDGEMENTS

The satisfaction that accompanies the successful completion of the project would be incomplete without the people who made it possible their constant guidance and encouragement crowned the efforts with success.

We express our deepest sense of gratitude to **Mr. V. BRAHMA NAIDU, M. Tech, Chairman**, Andhra engineering college, Atmakur, and other members of Management for providing all the facilities needed for this work.

We owe our gratitude to **Mr. K. VENUGOPAL, Secretary**, Andhra Engineering College, Atmakur, for his consistent help and valuable suggestions.

Our special thanks to **Dr.P. SIVA KRISHNA, PRINCIPAL**, Andhra Engineering College, Atmakur, for his timely suggestions and help during the progress of project work.

Our sincere special thanks to **Mr O. DEVANAND, Associate Professor & Head of the Department Computer Science & Engineering**, Andhra Engineering College, Atmakur, for his keen interest, critical, constructive and skillful guidance and constant encouragement throughout the course and for successful completion of project.

It is indeed our proud privilege to express our deep sense of gratitude and indebtedness to our guide, **Mr. D. V. SATHISH, Assistant Professor Computer Science & Engineering**, Andhra Engineering College, Atmakur, for his keen interest, critical, constructive and skillful guidance and constant encouragement throughout the course and for successful completion of project.

During the entire course of dissertation work, we received valuable academic inputs as well as moral support from other departments, general teaching and non-teaching faculty at Andhra Engineering College, Atmakur. We were motivated by the uphold and moral encouragement given to us by our beloved parents. Finally, we wish to express our sincere thanks for all those who helped me directly or indirectly to complete the work.

DECLARATION

We Ms. Sk. Manjunnisa (21HN1A0550), Ms. P. V. Meghana (21HN1A0565), Mr. Y. Pavan Kalyan (21HN1A0563), M s . V. Tanuja (21HN1A0561), M r . T. Mahendra (21HN1A0557) hereby declare that the main project report entitled **“VIDEO BASED ABNORMAL DRIVING BEHAVIOR DETECTION VIA DEEP-LEARNING FUSIONS”** is done by us under the Guidance of **Mr. M.SUDHAKAR**, M.Tech, Assistant professor of AIML." **ANDHRA ENGINEERING COLLEGE, NELLORE PALEM, ATMAKUR**" is Submitted in partial fulfilment of the requirements for the award of the Degree in **BACHELOR OF TECHNOLOGY**.

Project Associates

Ms. Sk. Manjunnisa	(21HN1A0550)
Ms. P. V. Meghana	(21HN1A0565)
Mr. Y. Pavan Kalyan	(21HN1A0563)
M s . V. Tanuja	(21HN1A0561)
M r . T. Mahendra	(21HN1A0557)

DATE:

PLACE :

VIDEO BASED ABNORMAL DRIVING BEHAVIOR
DETECTION VIA DEEP-LEARNING FUSIONS

TABLE OF CONTENTS

ABSTRACT	I
LIST OF FIGURES	II
LIST OF TABLES	-
LIST OF GRAPHS	-
LIST OF ABBREVIATIONS	III

S.NO	CONTENTS	PAGE NO
1	INTRODUCTION	1 - 3
2	LITERATURE SURVEY	4 - 8
3	SYSTEM REQUIREMENT SPECIFICATION	9 - 18
	3.1 Hardware Requirements	
	3.2 Software Requirements	
	3.3 Technology Description	
	3.4 Features of Python	
	3.5 Libraries Used	
4	SYSTEM ANALYSIS	19 – 23
	4.1 Existing Systems and its disadvantages	
	4.2 Proposed System and its advantages	
	4.3 Feasibility Study	

S.NO	CONTENTS	PAGE NO
5	SYSTEM DESIGN	24 - 31
	5.1 Work flow of the system	
	5.1.1 Activity Diagram	
	5.1.2 Sequence Diagram	
	5.1.3 Use Case Diagram	
	5.1.4 Class Diagram	
	5.1.5 Collaboration Diagram	
	5.2 UML Diagrams	
6	TESTING	32-36
7	CODING	37-43
8	RESULTS AND SCREENSHOTS	44-50
9	CONCLUSION	51-52
10	FUTURE WORK	53-54
11	BIBLIOGRAPHY	55-58

ABSTRACT

Video-based abnormal driving behavior detection is becoming more and more popular for the time being, as it is highly important in ensuring safeties of drivers and passengers in the vehicle, and it is an essential step in realizing automatic driving at the current stage. Thanks to recent developments in deep learning techniques, this challenging detection task can be largely facilitated via the prominent generalization capability of sophisticated deep learning models as well as large volumes of video clips which are indispensable for thoroughly training these data-driven deep learning models.

Deep learning fusion techniques are emphasized, and three novel deep learning-based fusion models inspired by the recently proposed and popular densely connected convolutional network (DenseNet) are introduced, to fulfill the video-based abnormal driving behavior detection task for the first time.

S.NO	LIST OF FIGURES	PAGE NO
1	3.3.1 Figure Neural Network	11
2	3.3.2 Figure Simple CNN Architecture	13
3	5.1.1 Figure Process Model/System Architecture	25
4	5.2.1 Figure Activity Diagram	27
5	5.2.2 Figure Sequence Diagram	28
6	5.2.3 Figure Usecase Diagram	29
7	5.2.4 Figure Class Diagram	30

S.NO	ACRONMY	ABREVIATION
1	Iso	International organization for Standardization
2	CNN	convolutional neural network
3	VOC	volatile organic compounds
4	VID	voltage Identification digital
5	I	form-and-color and-depth
6	VIPER	view, Interactor, presenter, entity, and Router
7	RNN	Recurrent neural network
8	LSTM	Long short-Term memory network
9	NLP	Natural Language processing
10	DFD	Data Flow Diagram
11	UML	unified modeling Language
12	AWGRD	Alternative wide groud residual densely network
13	WGD	wide group densely
14	IEEE	Institute of electrical and Cletronics engineers

CHAPTER – 1

INTRODUCTION

1. INTRODUCTION

It is widely acknowledged that, high-resolution videos are more and more commonly seen within a great number of visual applications at the current stage. For instance, in video surveillance, multiple high-resolution cameras are necessary to be placed at different locations. They work together to identify and track the moving target making the later high-level analyses based on the moving target (e.g., behavior or even potential intention) more feasible.

In emotional computation, high-resolution cameras need to be utilized to capture both obvious and fine changes of emotions of the target person in real-time which has significant impacts in security issues nowadays. It is easy to perceive from the above descriptions that, acquiring and storing a large volume of high-resolution videos are often not difficult to be realized for the time being.

However, the main challenge resides in how to efficiently and effectively make correct high-level decisions based on those low-level video clips of large volumes. In this study, high-resolution videos of drivers recorded within vehicles are emphasized.

The high-level decision here is to correctly detect abnormal driving behavior (i.e., patterns) of drivers. Automatic abnormal driving behavior detection is generally accepted as the first issue in realizing the popular fully autonomous driving task. It is certain that, for the autonomous driving task, safety issues are undoubtedly first priorities.

It is widely known that, behavior of drivers need to be well restricted in order to avoid any potential accident. Therefore, multiple high-resolution cameras equipped with in the driver's vehicle can be utilized to monitor the driver's status in real time.

Generally speaking, videos captured by high-resolution cameras also need to be processed immediately, in order to determine whether the current status of the driver is normal or not. It can be acknowledged from the above descriptions that, both the effectiveness (i.e., the detection accuracy) and the efficiency (i.e., the detection speed) of abnormal driving behavior detection are highly demanded.

Also, high-speed wireless transmissions are necessary to realize the swift and reliable transmission of high-quality videos, which further facilitates the above automatic abnormal driving behavior detection task.

In order to detect abnormal behavior of drivers, an official and precise definition of abnormal driving is often necessary. According to the International Organization for Standardization (ISO), abnormal driving is defined as the phenomenon that a driver's ability to drive is impaired due to her / his own focus on other activities unrelated to normal driving.

CHAPTER – 2

LITERATURE SURVEY

2. LITERATURE SURVEY

→ **FAST DEEP NEURAL NETWORKS WITH KNOWLEDGE GUIDED TRAINING AND PREDICTED REGIONS OF INTERESTS FOR REALTIME VIDEO OBJECT DETECTION**

AUTHORS: Jianhe Yuan, Wenming Cao, Zhihai He, Zhi Zhang

ABSTRACT: It has been recognized that deeper and wider neural networks are continuously advancing the state-of-the-art performance of various computer vision and machine learning tasks. However, they often require large sets of labeled data for effective training and suffer from extremely high computational complexity, preventing them from being deployed in real-time systems, for example vehicle object detection from vehicle cameras for assisted driving. In this paper, we aim to develop a fast deep neural network for real-time video object detection by exploring the ideas of knowledge-guided training and predicted regions of interest. Specifically, we will develop a new framework for training deep neural networks on datasets with limited labeled samples using cross-network knowledge projection which is able to improve the network performance while reducing the overall computational complexity significantly. A large pre-trained teacher network is used to observe samples from the training data. A projection matrix is learned to project this teacher-level knowledge and its visual representations from an intermediate layer of the teacher network to an intermediate layer of a thinner and faster student network to guide and regulate the training process. To further speed up the network, we propose to train a low-complexity object detection using traditional machine learning methods, such as support vector machine. Using this low-complexity object detector, we identify the regions of interest that contain the target objects with high confidence. We obtain a mathematical formula to estimate the regions of interest to save the computation for each convolution layer. Our experimental results on vehicle detection from videos demonstrated that the proposed method is able to speed up the network by up to 16 times while maintaining the object detection performance.

→ **CASCADED REGIONAL SPATIO-TEMPORAL FEATURE-ROUTING NETWORKS FOR VIDEO OBJECT DETECTION**

AUTHORS: Hui shuai, Qingshan Liu, Kaihua Zhang

ABSTRACT: This paper presents a cascaded regional spatiotemporal feature-routing networks for video object detection. Region proposal networks in faster region-based convolutional neural network (CNN) generate spatial proposals, whereas neglecting the temporal property of the videos. We incorporate the correlation filter tracking on the convolutional feature maps to explore an efficient and effective spatiotemporal region proposal generation method. To gradually refine the bounding boxes of proposals, three region classification and regression networks are cascaded. Feature maps from different layers in CNNs extract hierarchical information of the input, so we propose a router function which selects feature maps according to the scale of proposals. In addition, object co-occurrence inference is exploited to suppress conflicting false positives, which leads to a semantically coherent interpretation on the video. Extensive experiments on the Pascal VOC 2007 dataset and the ImageNet VID dataset show that the proposed method achieves the state-of-the-art performance for detecting unconstrained objects in cluttered scenes.

→ A NEUROMORPHIC PERSON RE-IDENTIFICATION FRAMEWORK FOR VIDEO SURVEILLANCE

AUTHORS: Aparajita Nanda, Pankaj Kumar Sa, Suman Kumar Choudhury, Sambit Bakshi

ABSTRACT: This paper presents a neuromorphic person re-identification (NPREId) framework to establish the correspondence among individuals observed across two disjoint camera views. The proposed framework comprises three modules (observation, cognition, and contemplation), inspired by the form-and-color-and-depth (I) theory model of object recognition system. In the observation module, a semantic partitioning scheme is introduced to segment a pedestrian into several logical parts, and an exhaustive set of experiments have been carried out to select the best possible complementary feature cues. In the cognition module, an unsupervised procedure is suggested to partition the gallery candidates into multiple consensus clusters with high intra-cluster and low inter-cluster similarity. A supervised classifier is then deployed to learn the relationship between each gallery candidate and its associated cluster, which is subsequently used to identify a set of inlier consensus clusters. This module also includes weighing of contribution of each feature channel toward defining a consensus cluster. Finally, in the contemplation module, the contributory weights are employed in a correlation-based similarity measure to find the corresponding match within the inlier set. The proposed framework is compared with several state-of-the-art methods on three challenging data sets: VIPeR, Ilids-VID, and CUHK01. The experimental results, with respect to recognition rates, demonstrate that the proposed framework can obtain superior performance as compared with the counterparts. The proposed framework, along with its low-rank bound property, further establishes its suitability in practical scenarios through yielding high cluster hit rate with low database penetration.

→ **SEMI-COUPLED DICTIONARY LEARNING WITH RELAXATION LABEL SPACE TRANSFORMATION FOR VIDEO-BASED PERSON RE- IDENTIFICATION**

AUTHORS: Lingchuan Sun, Zhuqing Jiang, Hongchao Song, Qishuo Lu

ABSTRACT: Video-based person reidentification (re-id) is a challenging problem due to much discrepancy between different videos by person pose, illumination, viewpoint change, background clutter, and occlusion within each camera and across different cameras. However, most existing video-based person re-id methods usually focus on dealing with the discrepancy between different cameras and do not fully consider the correlation between different cameras. In this paper, we propose a semicoupled dictionary learning with relaxation label space transformation approach to capture the intrinsic relationship of the same person under different cameras. First, to reduce the discrepancy between different views, we transform the original feature spaces into the common feature space by local Fisher discriminant analysis. Two dictionaries are learned from this common feature space. Second, a relaxation label space is introduced to associate the same person under different views. In this label space, the distance between different persons can be enlarged as much as possible, such that label information has stronger discriminative capability. A single dictionary is learned from the relaxation label space. Finally, in order to further enhance the correlation of the same person between different cameras, we use a pair of transformation matrices which map the coding coefficients learned from the common feature space to the coding coefficients learned from the relaxation label space, respectively. Extensive experimental results on two public Ilids Video re-Identification and Person Re-ID 2011 video-based person re-id datasets demonstrate the effectiveness of the proposed method.

CHAPTER-3

SYSTEM REQUIREMENTS

3. SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS:

- System : Pentium Dual Core.
- Hard Disk : 120 GB.
- Monitor : 15’’ LED
- Input Devices : Keyboard, Mouse
- Ram : 1 GB

3.2 SOFTWARE REQUIREMENTS:

- Operating system : Windows 10
- Coding Language : python
- Tool : PyCharm
- Database : MYSQL
- Server : Flask

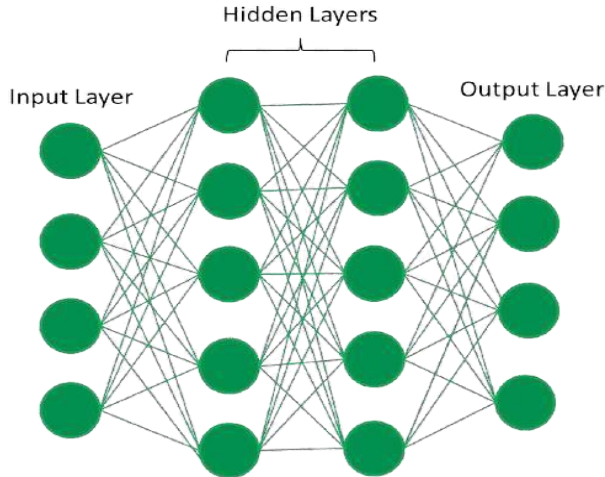
3.3 TECHNOLOGY DESCRIPTION: -

DEEP LEARNING: -

Deep Learning is transforming the way machines understand, learn, and interact with complex data. Deep learning mimics neural networks of the human brain, it enables computers to autonomously uncover patterns and make informed decisions from vast amounts of unstructured data.

How Deep Learning Works?

Neural network consists of layers of interconnected nodes, or neurons, that collaborate to process input data. In a fully connected deep neural network, data flows through multiple layers, where each neuron performs nonlinear transformations, allowing the model to learn intricate representations of the data. In a deep neural network, the input layer receives data, which passes through hidden layers that transform the data using nonlinear functions. The final output layer generates the model's prediction.



3.3.1 Fig(a): Neural Network

- Deep Learning:
 - GUI Applications (like TensorFlow, PyTorch etc.)
 - Web frameworks like Django (used by YouTube, Instagram, Dropbox)
 - Image processing (like OpenCV etc.)
 - Test frameworks

TYPES OF DEEP LEARNING: -

There are many different types of deep learning models. Some of the most common types include:

- **Convolutional neural networks (CNNs):**
CNNs are used for image recognition and processing.
- **Recurrent neural networks (RNNs):**
RNNs are used for natural language processing and speech recognition.
- **Long Short-Term Memory (LSTM) Networks:**
A type of RNN, LSTMs are particularly effective at capturing long-range dependencies in sequential data, making them useful for tasks like language translation.

❖ CNN MODEL:

A Convolutional Neural Network (CNN) is a type of deep learning algorithm specifically designed for processing and analyzing visual data, like images and videos, by using convolutional layers to extract features and patterns.

→ Neural Networks: Layers and Functionality

In a regular Neural Network, there are three types of layers:

1. **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data
2. **Hidden Layer:** The input from the Input layer is then fed into the hidden layer. There can be many hidden layers depending on our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features.
3. **Output Layer:** The output from the hidden layer is then fed into a logistic function like sigmoid or SoftMax which converts the output of each class into the probability score of each class.

CNN ARCHITECTURE:

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.

- **Convolutional Layer:**

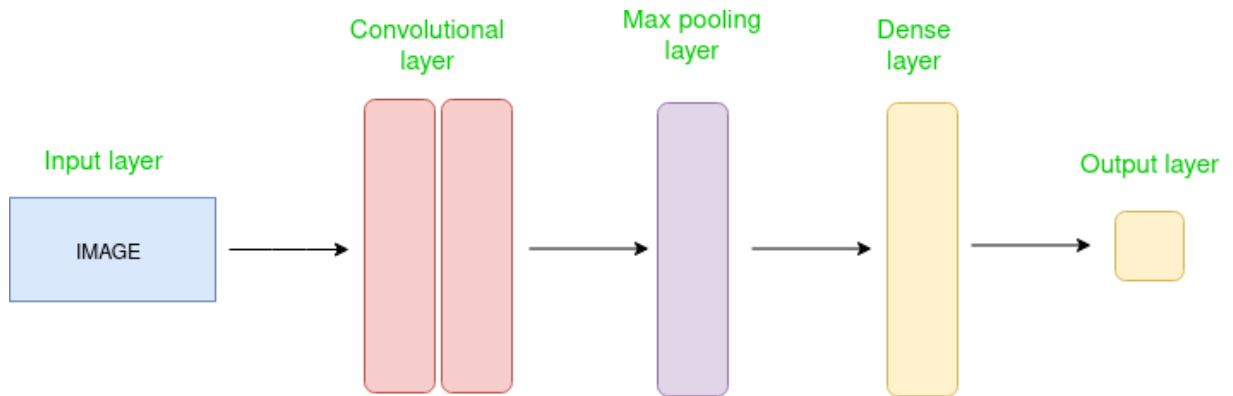
This layer applies filters (also known as kernels) to the input image to extract features like edges, textures, and patterns.

- **Pooling Layer:**

This layer reduces the spatial size of the feature maps generated by the convolutional layer, helping to make the network more robust to small variations in the input. Common pooling methods include max pooling and average pooling.

- **Fully Connected Layer:**

These layers connect every neuron in one layer to every neuron in the next layer, allowing the network to combine the learned features and make predictions.



3.3.2 Fig(b): Simple CNN architecture

TERMINOLOGIES USED IN DEEP LEARNING: -

- **Neural Network:** A computational model inspired by the human brain, consisting of interconnected nodes (neurons) organized in layers.
- **Neuron:** The basic unit of a neural network, receiving inputs, processing them, and producing an output.
- **Layers:** Organized groups of neurons, including input, hidden, and output layers.
- **Weights:** Numerical values assigned to connections between neurons, determining the strength of the connection.
- **Bias:** A constant value added to the weighted sum of inputs, allowing the neuron to activate even with zero input.
- **Activation Function:** A function applied to the output of a neuron, introducing non- linearity and enabling the network to learn complex patterns.

APPLICATIONS OF DEEP LEARNING:

Deep learning is used in various fields, including:

- Image Recognition
- Object Recognition and Speech Recognition
- Natural Language Processing (NLP)
- Fraud Detection
- Predictive Analytics
- Healthcare and Disease prediction
- Recommendation Systems
- Drug Discovery and Research

3.4 FEATURES OF PYTHON

Python stands out as a versatile, high-level programming language known for its readability, ease of use, and extensive libraries, making it suitable for various applications, including web development, and machine learning. Here's a breakdown of key features of Python

- **Free and Open Source:**

Python is free to use and distribute, and its source code is publicly available, fostering a strong community and collaboration.

- **Easy to Learn and Read:**

Python's syntax is designed to be simple and resemble natural language, making it easier for beginners to learn and for developers to read and maintain code.

- **Easy to code**

Python is very easy to learn the language as compared to other languages like C, C#, JavaScript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days.

- **High-Level Language:**

Python provides a high level of abstraction, allowing developers to focus on problem-solving rather than low-level details.

- **Interpreted Language:**

Python code is executed line by line, allowing for quick prototyping and testing.

- **Cross-Platform:**

Python code can run on various operating systems (Windows, macOS, Linux) without modification.

- **Extensible:**

Python can be extended with modules and packages, allowing developers to add new functionalities and integrate with other systems.

- **Dynamically Typed:**

Python's variables do not need to be declared with a specific type, making it flexible and easier to use.

- **Large Standard Library:**

Python comes with a rich set of built-in modules and packages, providing a wide range of functionalities without requiring external dependencies.

- **Multipurpose:**

Python can be used for a wide range of applications, including web development, data analysis, machine learning, scripting, and more.

- **Object-Oriented:**

Python supports object-oriented programming principles, allowing developers to create modular and reusable code.

- **Integrated Language**

Python is an integrated language because it can be readily connected with other programming languages like C, C++, Java, and more.

- **GUI Programming Support**

Python also provides an excellent support for the development of Graphical User Interfaces (GUIs) through various libraries and frameworks like TensorFlow, PyTorch, NumPy etc

3.5 Libraries Used in python

A Python library is a collection of related modules. It contains bundles of code that can be used repeatedly in different programs.

- **TensorFlow:**

This library was developed by Google in collaboration with the Brain Team. It is an open-source library used for high-level computations. It contains a large number of tensor operations.

- **Matplotlib:**

This library is responsible for plotting numerical data. It is also an open-source library and plots high-defined figures like pie charts, histograms, scatterplots, graphs, etc.

- **Pandas:**

Pandas are an important library for data scientists. It is an open-source machine learning library that provides flexible high-level data structures and a variety of analysis.

- **NumPy:**

The name “NumPy” stands for “Numerical Python”. It is the commonly used library. It is a popular machine learning library that supports large matrices and multi-dimensional data. It consists of in- built mathematical functions for easy computations.

- **Keras:**

Keras is a high-level, open-source Python library for building and training neural networks, designed to simplify the process of working with deep learning models and enabling fast experimentation

- **OpenCV Python**

Open-Source Computer Vision or OpenCV is used for image processing. It is a Python package that monitors overall functions focused on instant computer vision.

- **Scikit-learn:**

Scikit-learn is an open-source library that supports machine learning. It supports various supervised and unsupervised algorithms like linear regression, classification, clustering, etc. This library works in association with NumPy and SciPy.

- **PyTorch:**

PyTorch is the largest machine learning library that optimizes tensor computations. It has rich APIs to perform tensor computations with strong GPU acceleration.

CHAPTER – 4

SYSTEM ANALYSIS

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM:

- Abnormal driving detection and deep learning techniques, which are closely related to this study, are emphasized. Recent developments in the two aspects are briefly reviewed, with pros and cons been discussed.
- Can be summarized based on literatures of automatic abnormal driving behavior detection that, there are often three commonly used detection schemes.
- The first one is based on the detection of human physiological signals (i.e., electrooculogram, electro-encephalogram, respiratory, blood flow changes, etc.) using diverse kinds of sensors . The second one is based on facial details [29] (i.e., changes in eye movement, mouth movement, head movement, hand features, etc.).
- Deep learning techniques receive vast popularity when powerful computational hardware and large-scale data become more and more available nowadays.
- Generally speaking, most contemporary deep learning models can be categorized into two types, i.e., deep generative learning models and deep discriminant learning models. To be specific, deep generative learning models mainly aim to replicate “fake-but-realistic” data based on real data, and popular deep generative learning models include but not limited to VAE (i.e., variational auto-encoder) [34], GAN (i.e., generative adversarial network) [35], GLOW (i.e., generative flow)

DISADVANTAGES OF EXISTING SYSTEM:

- Intrusive and uncomfortable: Requires the driver to wear sensors (e.g., EEG, ECG, EOG) which can be intrusive or uncomfortable.
- High cost: Specialized sensors and equipment increase system cost.
- Sensitive to noise: Physiological signals can be affected by motion artifacts, environmental noise, or sensor misplacement.
- Personal calibration required: Signals vary from person to person, often requiring individual calibration.

- Limited real-world practicality: Difficult to implement on a large scale in consumer vehicles.

4.2 PROPOSED SYSTEM:

- The proposed deep learning-based fusion models in automatically detecting abnormal driving behavior of this study, the Kaggle state farm distracted driver detection database
- demonstrates the trend of accuracies increasing with respect of training epochs in all compared deep learning models. First, it can be noticed that, accuracies of all deep learning models keep on increasing and then become stable when their training epochs further increase, which is a significant indicator of the thorough training and convergence of all deep learning models. Second, three deep learning-based fusion models, DenseNet, as well as ResNet outperform other conventional CNN-based models (i.e., CNN, Wide CNN, Group CNN) as revealed in Figure 9. For comparisons between three deep learning-based fusion models and DenseNet, it is interesting to notice that, the former reaches the stable stage faster (i.e., less epochs) than DenseNet, and significant robustness can be obtained from new deep learning- based fusion models.

ADVANTAGES OF PROPOSED SYSTEM:

- The main advantage of affecting drivers' normal drivings cannot be neglected, either. Furthermore, physiological signals of human beings vary greatly due to the physiological difference in each individual person and her / his environmental conditions

4.3 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. Feasibility analysis is a study that examines the practicality and potential success of a proposed project or system. It helps to identify potential problems, opportunities, and solutions.

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY:

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY:

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

- **Hardware Requirements:** High-performance computing hardware (e.g. GPUs. TPUs) is required for training and deploying CNN models.

- **Software Requirements:** Open-source deep learning frameworks (e.g. TensorFlow, PyTorch) and libraries (e.g. OpenCV) are available for building and deploying CNN models.

SOCIAL FEASIBILITY:

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER – 5

SYSTEM DESIGN

5. SYSTEM DESIGN

5.1 Work Flow of a System:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

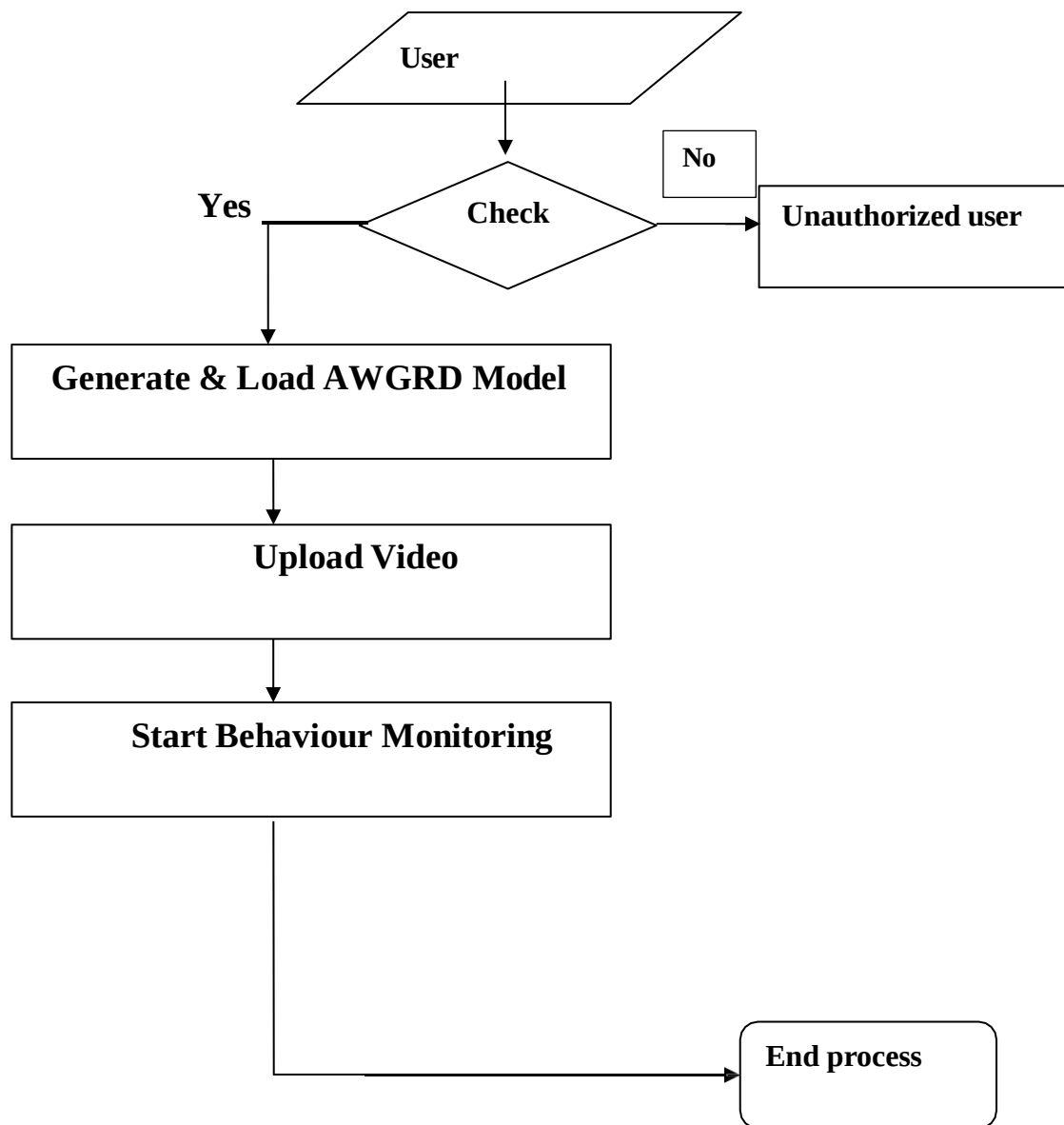


Fig : System Architecture

5.2 UML DIAGRAMS: -

- UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.
- The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.
- The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non- software systems.
- The UML is a very important part of developing object-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

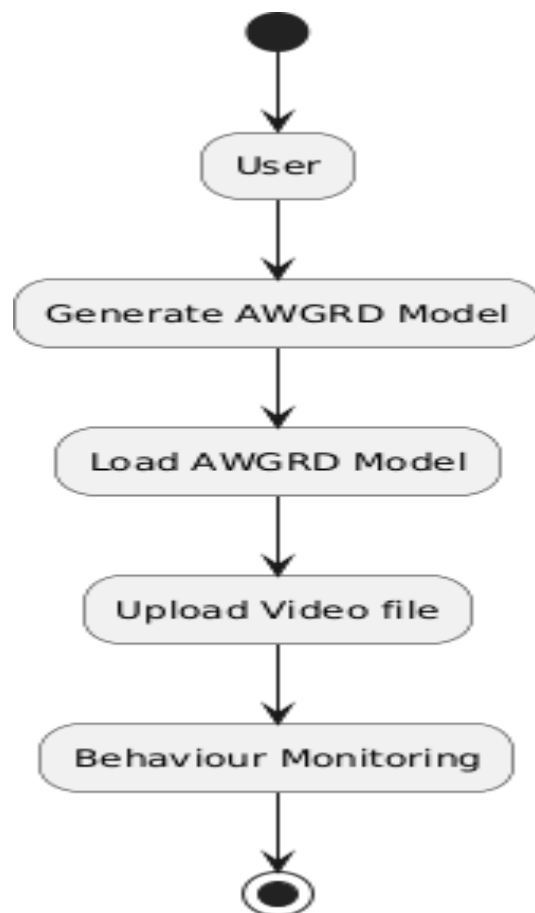
GOALS: -

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

5.2.1 ACTIVITY DIAGRAM

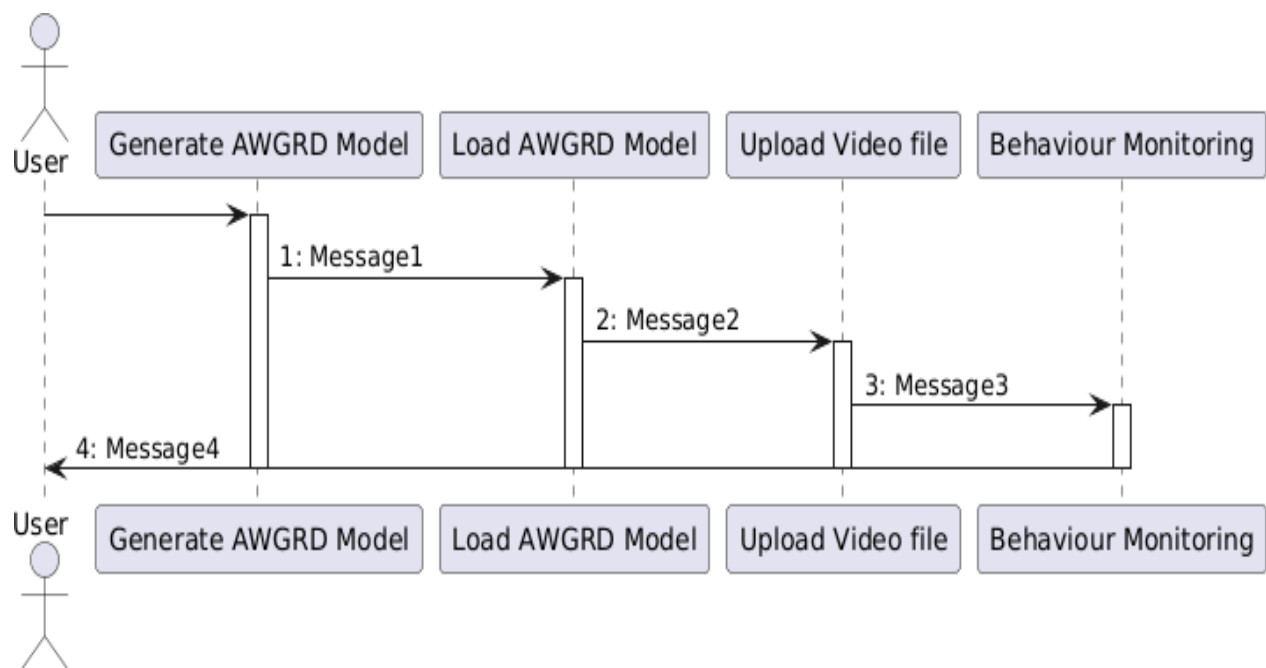
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



5.2.1 Fig(a) : Activity Diagram

5.2.2 SEQUENCE DIAGRAM

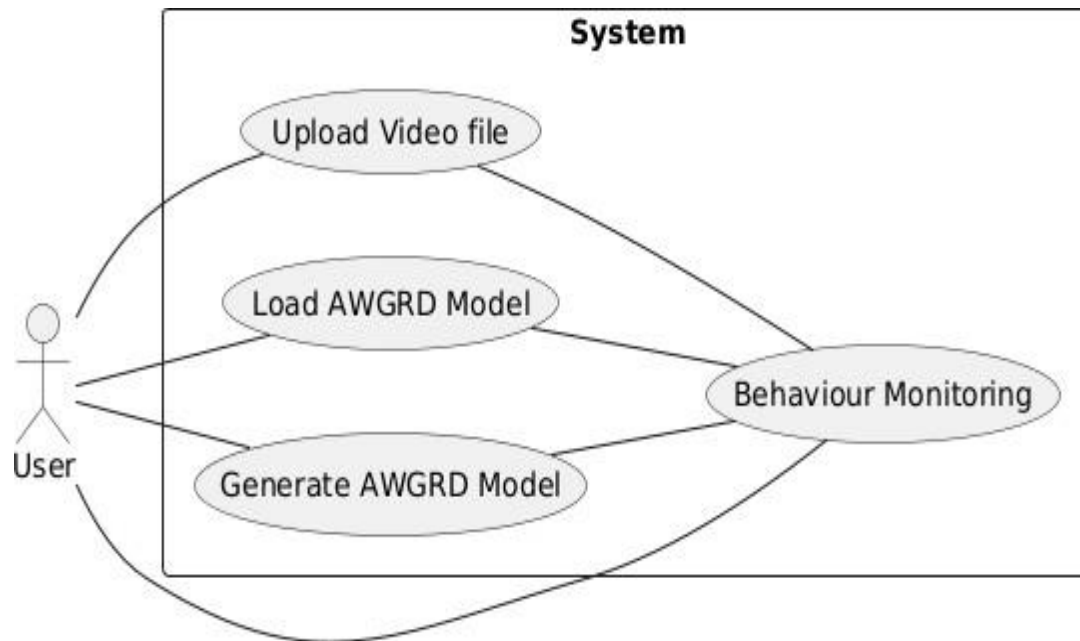
A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



5.2.2 Fig(b) : Sequential Diagram

5.2.3 USE CASE DIAGRAM

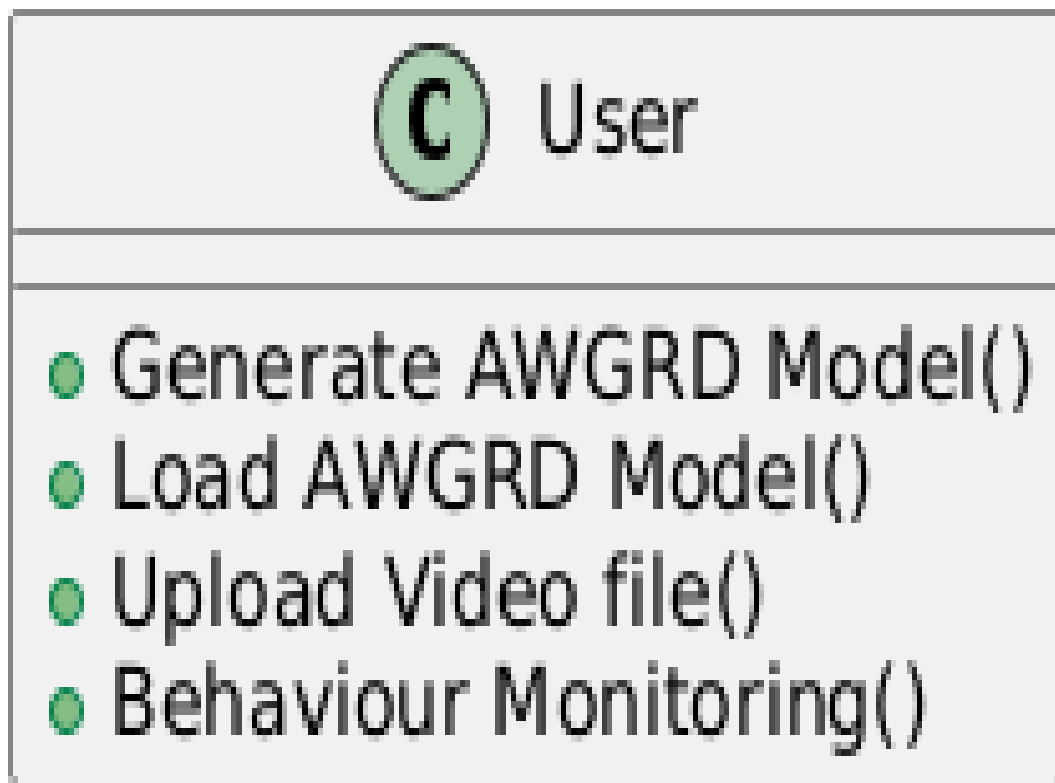
A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



5.2.3 Fig (c): Use Case Diagram

5.2.4 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



5.2.4 Fig(d) : Class Diagram

CHAPTER – 6

TESTING

6. TESTING

6.1 TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

TYPES OF TESTS

✓ UNIT TESTING:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

✓ INTEGRATION TESTING:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

✓ **FUNCTIONAL TESTING:**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

➤ Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

SYSTEMS/PROCEDURES:

Interfacing systems or procedures must be involved.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

✓ **SYSTEM TESTING:**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

✓ **WHITE BOX TESTING:**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

✓ **BLACK BOX TESTING:**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. You cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

✓ **FIELD TESTING:**

Field testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

TEST OBJECTIVES:

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

FEATURES TO BE TESTED:

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

✓ **INTEGRATION TESTING:**

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications,

e.g. components in a software system or – one step up – software applications at the company level – interact without error

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

✓ **ACCEPTANCE TESTING:**

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER – 7

CODING

CODING

Source Code:

```
import torch

from fusion_model import DrivingFusionModel from utils import load_video_frames

device = torch.device("cuda" if torch.cuda.is_available() else "cpu") model =
DrivingFusionModel()
model.load_state_dict(torch.load("driving_model.pth", map_location=device))
model.to(device)
model.eval()

def predict(video_path):
    video = load_video_frames(video_path).unsqueeze(0).to(device) # [1, T, C, H, W]

    with torch.no_grad(): output = model(video)

    pred = torch.argmax(output, dim=1).item() return "Abnormal" if pred == 1 else
    "Normal"

# Example
video_file = "dataset/abnormal/video2.mp4" result = predict(video_file)
print(f'Prediction: {result}')
```

```
import os import torch

from torch.utils.data import Dataset, DataLoader from fusion_model import
DrivingFusionModel from utils import load_video_frames

class DrivingDataset(Dataset):
def __init__(self, root, num_frames=16): self.samples = []
    self.labels = []
    self.num_frames = num_frames
    for label, cls in enumerate(['normal', 'abnormal']): folder = os.path.join(root, cls)
    for file in os.listdir(folder): if file.endswith('.mp4'):
        self.samples.append(os.path.join(folder, file)) self.labels.append(label)
```



```
def _len_(self):
    return len(self.samples)

def __getitem__(self, idx):
    video = load_video_frames(self.samples[idx], self.num_frames) label = self.labels[idx]
    return video, label

# Training
device = torch.device("cuda" if torch.cuda.is_available() else "cpu") dataset =
DrivingDataset("dataset")
loader = DataLoader(dataset, batch_size=4, shuffle=True) model =
DrivingFusionModel().to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-4) criterion =
torch.nn.CrossEntropyLoss()

for epoch in range(10): total_loss = 0 model.train()
for videos, labels in loader: videos = videos.to(device)
    labels = torch.tensor(labels).to(device) outputs = model(videos)
    loss = criterion(outputs, labels)

optimizer.zero_grad() loss.backward()
```

```
optimizer.step() total_loss += loss.item()
```

```
print(f'Epoch {epoch+1} – Loss: {total_loss:.4f}')
```

```
torch.save(model.state_dict(), "driving_model.pth")
```

```
import cv2 import torch
```

```
from torchvision import transforms import os
```

```
def load_video_frames(path, num_frames=16): cap = cv2.VideoCapture(path)
```

```
total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) step = max(total // num_frames, 1)
```

```
transform = transforms.Compose([ transforms.ToPILImage(), transforms.Resize((224,  
224)), transforms.ToTensor()])
```

```
frames = []
```

```
for I in range(num_frames): cap.set(cv2.CAP_PROP_POS_FRAMES, I * step)
```

```
ret, frame = cap.read() if not ret:
    break
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) frame = transform(frame)
frames.append(frame)

cap.release()
while len(frames) < num_frames: frames.append(torch.zeros_like(frames[0]))
    return torch.stack(frames) # [T, C, H, W]

import torch
import torch.nn as nn
import torchvision.models as models

class CNNEncoder(nn.Module): def _init_(self):
    super(CNNEncoder, self)._init_()
    resnet = models.resnet18(pretrained=True)
    self.features = nn.Sequential(*list(resnet.children())[:-1]) # Remove FC

def forward(self, x): x = self.features(x)
    return x.view(x.size(0), -1)
```

```
class DrivingFusionModel(nn.Module):
def _init_(self, feature_dim=512, hidden_dim=128, num_classes=2):
    super(DrivingFusionModel, self)._init_()
    self.cnn = CNNEncoder()
    self.lstm = nn.LSTM(input_size=feature_dim, hidden_size=hidden_dim, batch_first=True)
    self.fc = nn.Linear(hidden_dim, num_classes)

def forward(self, x_seq): # [B, T, C, H, W] B, T, C, H, W = x_seq.size()
    features = []
    for t in range(T):
        f = self.cnn(x_seq[:, t]) features.append(f)
    features = torch.stack(features, dim=1) # [B, T, 512] lstm_out, _ = self.lstm(features)
    out = self.fc(lstm_out[:, -1, :]) # Use last timestep return out
```

CHAPTER – 8

EXPECTED RESULT

8 EXPECTED RESULT

8.1 RESULTS

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.
2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.
3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

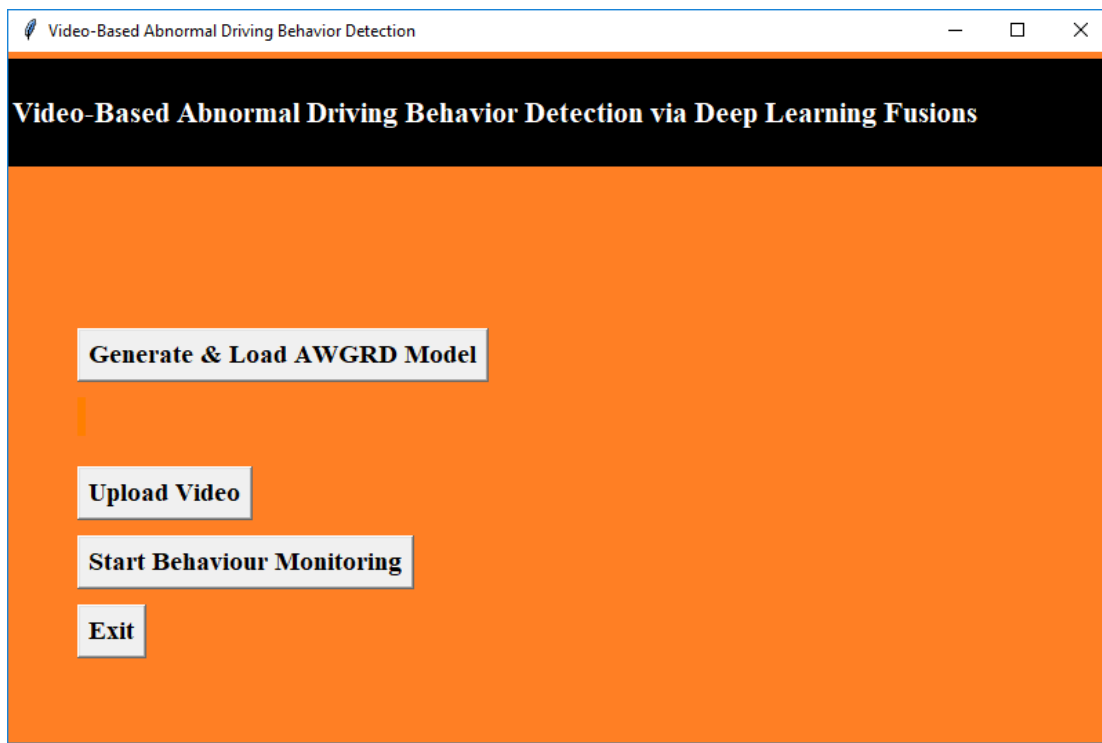
A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
2. Select methods for presenting information.
3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

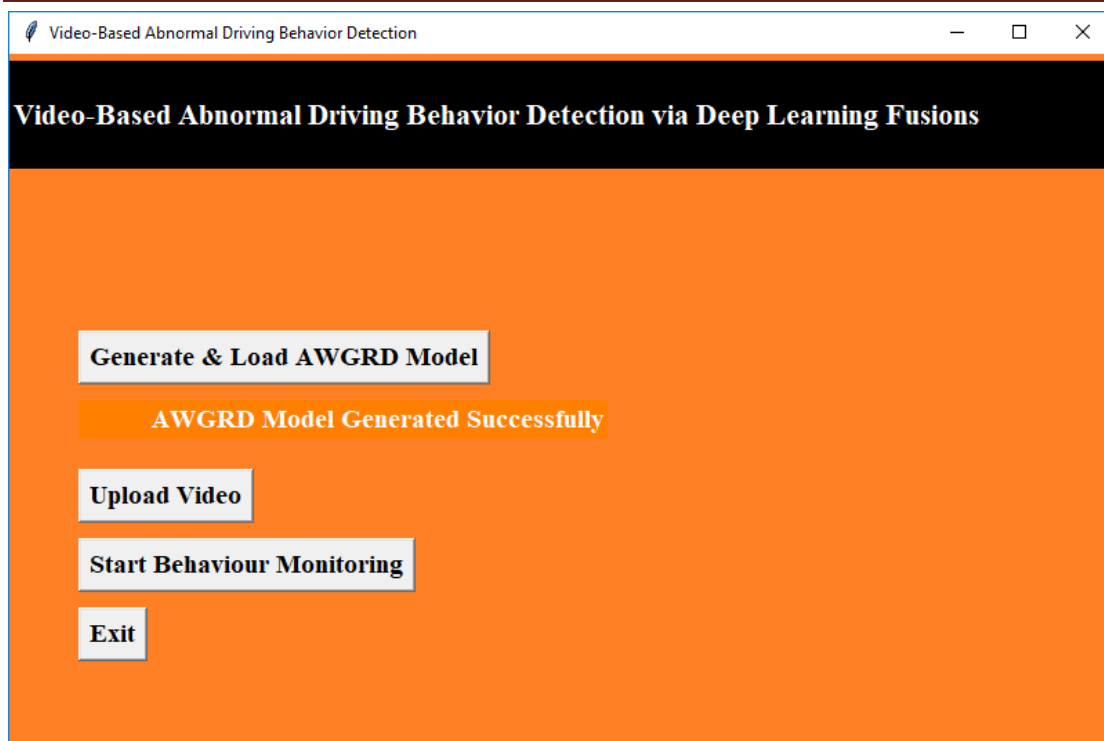
- ❖ Convey information about past activities, current status or projections of the
- ❖ Future.
- ❖ Signal important events, opportunities, problems, or warnings.
- ❖ Trigger an action.
- ❖ Confirm an action.

8.2 SCREENSHOTS



Fig(a): Generate AWGRD Model

In above screen click on 'Generate & Load AWGRD Model' button to generate AWGRD train model. All model information we can see in black console after clicking on button



Fig(b): AWGRD Model generate successfully

In above screen we can see model is generated and in below screen we can see all details

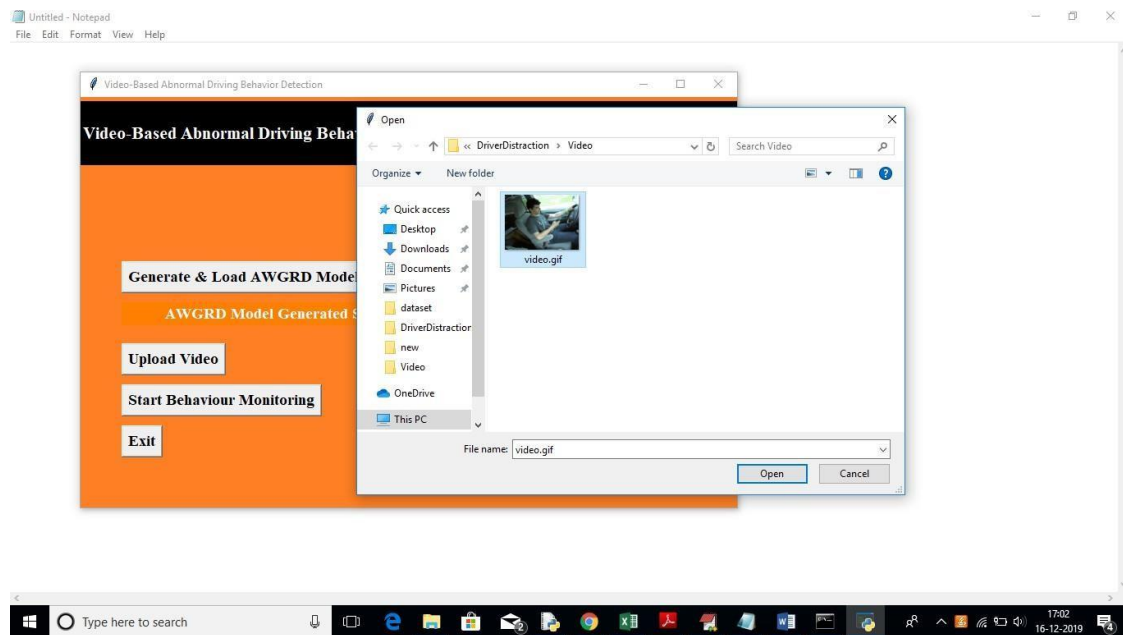
```

C:\Windows\system32\cmd.exe
AbnormalBehaviour.py:40: UserWarning: Update your `Dense` call to the Keras 2 API: `Dense(activation="relu", units=128)`
  awgrd_model.add(Dense(output_dim = 128, activation = 'relu'))
AbnormalBehaviour.py:41: UserWarning: Update your `Dense` call to the Keras 2 API: `Dense(activation="softmax", units=10)`
  awgrd_model.add(Dense(output_dim = 10, activation = 'softmax'))
Model: "sequential_1"

Layer (type)                 Output Shape              Param #
-----
conv2d_1 (Conv2D)            (None, 148, 148, 32)      896
max_pooling2d_1 (MaxPooling2 (None, 74, 74, 32)        0
conv2d_2 (Conv2D)            (None, 72, 72, 32)        9248
max_pooling2d_2 (MaxPooling2 (None, 36, 36, 32)        0
flatten_1 (Flatten)          (None, 41472)             0
dense_1 (Dense)              (None, 128)               5308544
dense_2 (Dense)              (None, 10)                1290
-----
Total params: 5,319,978
Trainable params: 5,319,978
Non-trainable params: 0
None
    
```

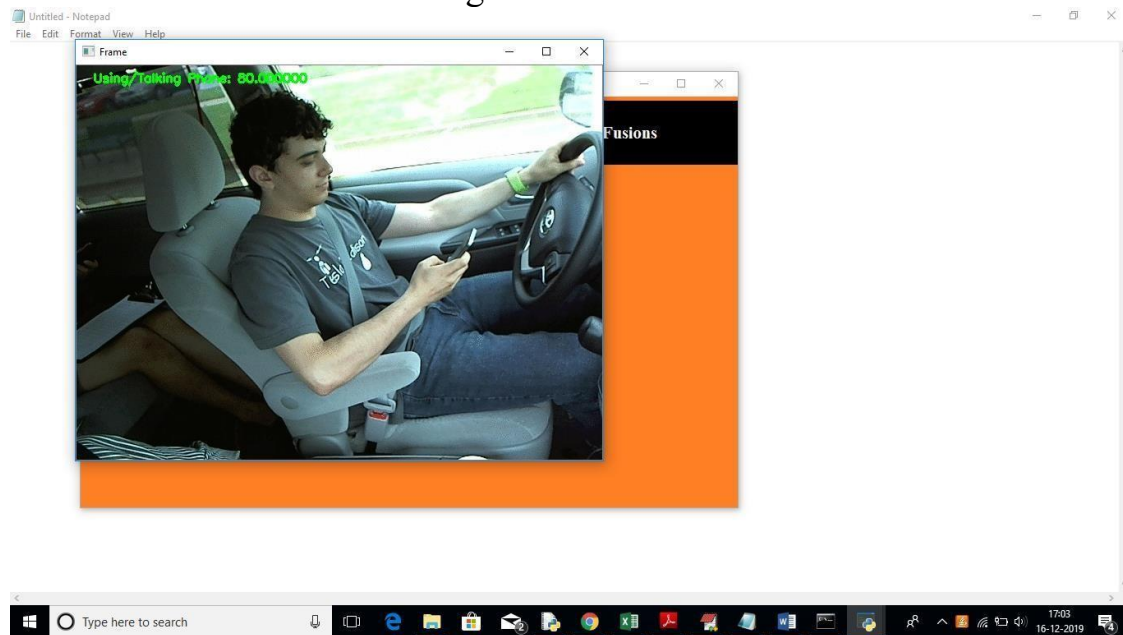
Fig(c): upload video

In above screen dense model using AWGRD is created. Now click on ‘Upload Video’ button to upload video



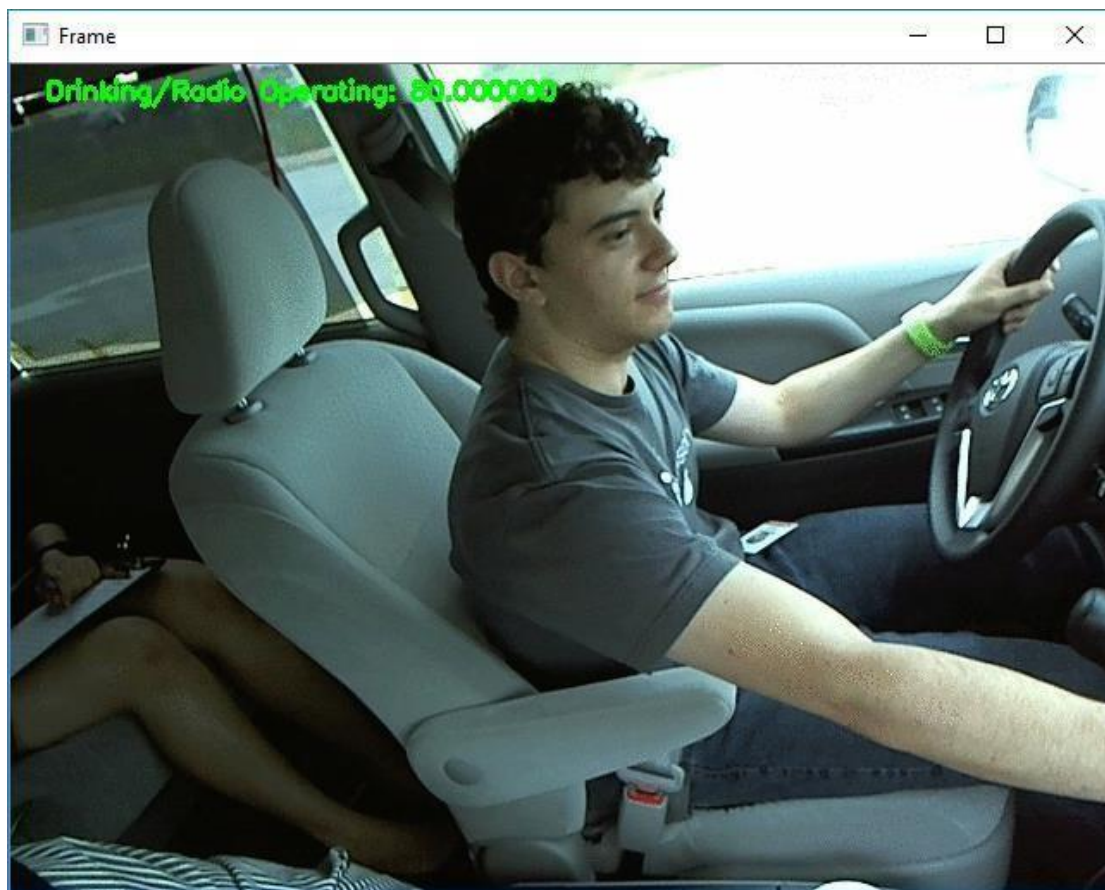
Fig(d): start behavior monitor

In above screen uploading one video and then click on open button and then click on 'Start Behaviour Monitoring' button



Fig(e): Detect user is using phone

In above screen application detected user is using/talking on phone



Fig(f): Detection performed

In above screen we can see user is try to start Radio. Similarly other detection will also be performed.

CHAPTER-9

CONCLUSION

9 CONCLUSION

The video-based abnormal driving behavior detection study is highly important nowadays, as it is a reliable and automatic manner to ensure safeties of drivers. Also, it receives vast popularity as it is an essential step to realize fully automatic driving (i.e., particularly in Level-3 and Level-4 stages according to the “autonomous driving” definition provided by the US Department of Transportation’s National Highway Traffic Safety Administration). In this study, three novel deep learning-based fusion models are introduced for the first time, to fulfill the video-based abnormal driving behavior detection task. Technically, these new models are inspired by the popular DenseNet, which was proposed in recent years. For WGD, it emphasizes on important issues of designs of modern deep learning models, including the depth, the width, and the cardinality. The width and the cardinality of WGD significantly increase, therein. For WGRD and AWGRD, they are more sophisticated as the important idea of residual networks with superpositions of previous layers is incorporated. This idea is highly valuable in the video-based abnormal driving behavior detection task, as temporary and spatial latent information can be comprehensively described with the help of superpositions of previous layers. Extensive experiments based on the standard Kaggle state farm distracted driver detection dataset as well as rigorous comparisons with several other popular deep learning models suggest the superiority of newly proposed deep learning-based fusion models in both effectiveness and efficiency.

CHAPTER-10

FUTURE WORK

10 FUTURE WORK

Future Work is being planned to further analyze and enhance the protocol issues author has describe 3 algorithms based on CNN deep learning models.

- 1) Wide Group Densely (WGD) Network: Technically, WGD takes important issues of deep learning models, i.e., the depth, the width and the cardinality, into consideration when designing its model structure based on Dense Net. This model use deep features from input train model to get better prediction accuracy.
- 2) Wide Group Residual Densely (WGRD) Network: The most significant change of WGRD with respect to WGD is that, the idea of residual networks is incorporated in WGRD. In this algorithm input image will pass from one layer to other residual layer to have best features from train input image to get best accuracy.
- 3) Alternative Wide Group Residual Densely (AWGRD) Network: This algorithm works similar to above two algorithms but while passing input data from one layer to other, this algorithm will take super positions of previous layers which has best features from all layer and will have better prediction accuracy. Due to super positions extraction training efficiency will undoubtedly become higher.

In the future, “global + local” latent features-based abnormal driving behavior detection studies will be conducted. Also, effective and efficient deep learning models realized via customized mobile chips will be investigated for realizing abnormal driving behavior detection.

CHAPTER-11

BIBLIOGRAPHY

11 BIBLIOGRAPHY

- [1] W. Cao, J. Yuan, Z. He, Z. Zhang, and Z. He, “Fast deep neural networks with knowledge guided training and predicted regions of interests for realtime video object detection,” IEEE Access, vol. 6, pp. 8990–8999, 2018.
- [2] H. Shuai, Q. Liu, K. Zhang, J. Yang, and J. Deng, “Cascaded regional spatio-temporal feature-routing networks for video object detection,” IEEE Access, vol. 6, pp. 3096–3106, 2018.
- [3] A. Nanda, P. K. Sa, S. K. Choudhury, S. Bakshi, and B. Majhi, “A neuromorphic person re-identification framework for video surveillance,” IEEE Access, vol. 5, pp. 6471–6482, 2017.
- [4] L. Sun, Z. Jiang, H. Song, Q. Lu, and A. Men, “Semi-coupled dictionary learning with relaxation label space transformation for video-based person re-identification,” IEEE Access, vol. 6, pp. 12587–12597, 2018.
- [5] Y. Wu, Y. Sui, and G. Wang, “Vision-based real-time aerial object localization and tracking for UAV sensing system,” IEEE Access, vol. 5, pp. 23969–23978, 2017.
- [6] S.-H. Lee, M.-Y. Kim, and S.-H. Bae, “Learning discriminative appearance models for online multi-object tracking with appearance discriminability measures,” IEEE Access, vol. 6, pp. 67316–67328, 2018.
- [7] M. S. Hossain and G. Muhammad, “An emotion recognition system for mobile applications,” IEEE Access, vol. 5, pp. 2281–2287, 2017.
- [8] Z. Pan, X. Yi, and L. Chen, “Motion and disparity vectors early determination for texture video in 3D- HEVC,” Multimedia Tools Appl., to be published. doi: 10.1007/s11042-018-6830-7.
- [9] J. Wang, Z. Zhang, B. Li, S. Lee, and R. S. Sherratt, “An enhanced fall detection system for elderly person monitoring using consumer home networks,” IEEE Trans. Consum. Electron., vol. 60, no. 1, pp. 23–29, Feb. 2014.
- [10] Z. Zhang, X. Guo, and Y. Lin, “Trust management method of D2D communication based on RF fingerprint identification,” IEEE Access, vol. 6, pp. 66082–66087, 2018.
- [11] J. Wang, Y. Cao, B. Li, H.-J. Kim, and S. Lee, “Particle swarm optimization based

- clustering algorithm with mobile sink for WSNs,” *Future Gener. Comput. Syst.*, vol. 76, pp. 452–457, Nov. 2017. [12] Z. Xue, J. Wang, G. Ding, Q. Wu, Y. Lin, and T. A. Tsiftsis, “Deviceto-device communications underlying UAV- supported social networking,” *IEEE Access*, vol. 6, pp. 34488–34502, 2018.
- [13] J. Wang, J. Cao, R. S. Sherratt, and J. H. Park, “An improved ant colony optimization-based approach with mobile sink for wireless sensor networks,” *J. Supercomput.*, vol. 74, no. 12, pp. 6633–6645, Dec. 2018.
- [14] Y. Tu, Y. Lin, J. Wang, and J.-U. Kim, “Semi-supervised learning with generative adversarial networks on digital signal modulation classification,” *Comput. Mater. Continua*, vol. 55, no. 2, pp. 243–254, 2018.
- [15] J. Wang, Y. Gao, X. Yin, F. Li, and H.-J. Kim, “An enhanced PEGASIS algorithm with mobile sink support for wireless sensor networks,” *Wireless Commun. Mobile Comput.*, vol. 2018, Dec. 2018, Art. no. 9472075.
- [16] J. Sun et al., “A multi-focus image fusion algorithm in 5G communications,” *Multimedia Tools Appl.*, vol. 3, pp. 1–20, Feb. 2018.
- [17] J. Wang, C. Ju, Y. Gao, A. K. Sangaiah, and G.-J. Kim, “A PSO based energy efficient coverage control algorithm for wireless sensor networks,” *Comput. Mater. Continua*, vol. 56, no. 3, pp. 433–446, Sep. 2018.
- [18] Y. Lin, X. Zhu, Z. Zheng, Z. Dou, and R. Zhou, “The individual identification method of wireless device based on dimensionality reduction and machine learning,” *J. Supercomput.*, vol. 5, pp. 1–18, Dec. 2017.
- [19] E. B. Tirkolaei, A. A. R. Hosseinabadi, M. Soltani, A. K. Sangaiah, and J. Wang, “A hybrid genetic algorithm for multi-trip green capacitated arc routing problem in the scope of urban services,” *Sustainability*, vol. 10, no. 5, p. 1366, 2018.
- [20] W. Qidi, L. Yibing, L. Yun, and Y. Xiaodong, “The nonlocal sparse reconstruction algorithm by similarity measurement with shearlet feature vector,” *Math. Problems Eng.*, vol. 2014, Mar. 2014, Art. no. 586014.
- [21] Q. Wu, Y. Li, and Y. Lin, “The application of nonlocal total variation in image denoising for mobile transmission,” *Multimedia Tools Appl.*, vol. 76, no. 16, pp. 17179–17191, Aug. 2016.
- [22] D. Zhang, D. Meng, and J. Han, “Co-saliency detection via a selfpaced multiple-

- instance learning framework,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 5, pp. 865–878, May 2017.
- [23] D. Zeng, Y. Dai, F. Li, R. S. Sherratt, and J. Wang, “Adversarial learning for distant supervised relation extraction,” Comput. Mater. Continua, vol. 55, no. 1, pp. 121–136, 2018.
- [24] R. Olson, R. Hanowski, J. Hickman, and J. Bocanegra, “Driver distraction in commercial vehicle operations,” U.S. Dept. Transp., Rep. Federal Motor Carrier Saf. Admin., Washinton, DC, USA, Tech. Rep. FMCSA-RRT-09- 042, 2009.
- [25] G. Huang, Z. Liu, L. Van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in Proc. CVPR, Jul. 2017, pp. 4700–4708.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. CVPR, Jun. 2016, pp. 770–778.
- [27] A. Saeed, S. Trajanovski, M. Van Keulen, and J. Van Erp, “Deep physiological arousal detection in a driving simulator using wearable sensors,” in Proc. ICDMW, Nov. 2017, pp. 486–493.
- [28] Z. Pan, H. Qin, X. Yi, Y. Zheng, and A. Khan, “Low complexity versatile video coding for traffic surveillance system,” Int. J. Sensor Netw., vol. 30, no. 2, pp. 116–125, 2019.
- [29] M. Jeong, B. C. Ko, S. Kwak, and J.-Y. Nam, “Driver facial landmark detection in real driving situations,” IEEE Trans. Circuits Syst. Video Technol., vol. 28, no. 10, pp. 2753–2767, Oct. 2018.
- [30] V. Balasubramanian and R. Bhardwaj, “Grip and electrophysiological sensor-based estimation of muscle fatigue while holding steering wheel in different positions,” IEEE Sensors J., vol. 19, no. 5, pp. 1951– 1960, Mar. 2019.